

Basi di dati

Maurizio Lenzerini

Nome autore

Università degli studi La Sapienza

Ingegneria Informatica e Automatica - 3 anno

Gennaio - Dicembre 2026-27

Indice

1	Modello Relazionale	2
1.1	Relazione matematica	2
	Notazioni	3
	Schema di relazione e tupla etichettata	3
	Schemi e istanze	4
	Algebra relazionale	4
	Operatori insiemistici	4
	Ridenominazione	6
	selezione	7
	Proiezione	8
	Selezione e Proiezione	8
	La negazione in algebra relazionale	9
	Join	9
	join naturale	9
	Join completo	10
	Theta-join	10
	Self-join	11
	Riassunto join	11
	Esercizi query	11
2	SQL	13

Capitolo 1

Modello Relazionale

1.1 Relazione matematica

Siano $D_1, D_2, \dots, D_n$ n insiemi:

Il **prodotto cartesiano** $D_1 \times D_2 \times \dots \times D_n$ è l'insieme di tutte le n -uple ordinate $(d_1, d_2, \dots, d_n)$ tali che $d_1 \in D_1, d_2 \in D_2, \dots, d_n \in D_n$. Una relazione matematica sugli insiemi $D_1, D_2, \dots, D_n$ è un **sottoinsieme del prodotto cartesiano** $D_1 \times D_2 \times \dots \times D_n$.

Se R è una relazione matematica sugli insiemi $D_1, D_2, \dots, D_n$ tali insiemi sono detti domini, tale relazione su n domini dice che ha grado n . il numero di n -uple (righe) di una relazione è la cardinalità della relazione stessa.

Esempio:

- $D_1 = \{\text{Conti, Totti, Maradona}\}$
- $D_2 = \{\text{Genoa, Roma}\}$
- prodotto cartesiano $D_1 \times D_2$

Conti	Genoa
Conti	Roma
Totti	Genoa
Totti	Roma
Maradona	Genoa
Maradona	Roma

- una relazione: $\text{HaGiocato} \subseteq D_1 \times D_2$

Conti	Roma
Conti	Genoa
Totti	Roma

La relazione matematica è quindi un insieme di ennuple o tuple ordinate su $D_1, D_2, \dots, D_n$, dove ogni tupla è una sequenza di valori della forma $(d_1, \dots, d_n)$. Ciascuna tupla è ordinata nel senso che il valore del primo elemento della tupla proviene dal primo dominio, ma non è definito un ordinamento tra le tuple e sono tutte distinte.

Una relazione del modello relazione si può quindi rappresentare come una tabella in cui gli attributi sono usati come intestazioni delle colonne. La struttura non è posizionale cioè l'ordinamento delle colonne nella tabella è irrilevante. Ogni riga della tabella corrisponde ad una tupla della relazione, e siccome la relazione è un insieme di tuple, non ci possono essere due righe uguali nella tabella.

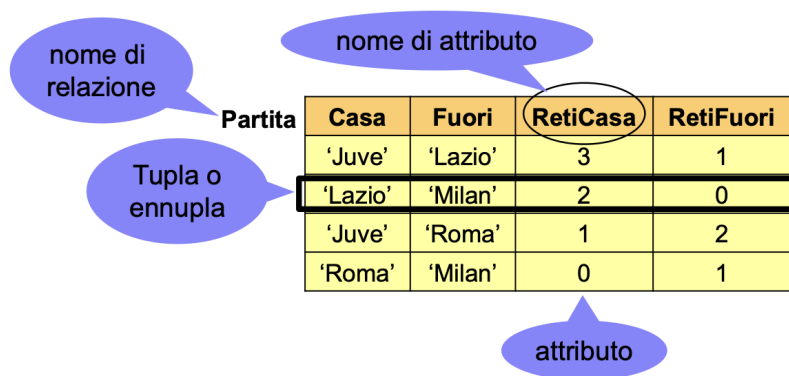


Figura 1.1: caption

Gli attributi devono essere atomici cioè non devono essere un array ma un singolo valore, e non ci devono essere due tuple uguali.

Notazioni

Sia X l'insieme degli attributi di una relazione R . Se t è una tupla di R , cioè una tupla su X , e se $A \in X$ è un attributo in X , allora $t[A]$ oppure anche $t.A$ indica il valore che la tupla t ha in corrispondenza dell'attributo A .
 esempio: $t[Fuori] = Lazio$ $t[Fuori, RetiFuori] = \langle Lazio, 1 \rangle$ $t[] = \text{tupla vuota}$.

Partita	Casa	Fuori	RetiCasa	RetiFuori
t1 →	'Juve'	'Lazio'	3	1
t2 →	'Lazio'	'Milan'	2	0
t3 →	'Juve'	'Roma'	1	2
	'Roma'	'Milan'	0	1

$t1[Fuori] = \text{'Lazio'}$
 $t2[RetiCasa] = 2$
 $t3[Casa, RetiCasa] = \langle \text{'Juve'}, 1 \rangle$

Figura 1.2: caption

Schema di relazione e tupla etichettata

La specifica del nome R di una relazione, degli attributi $A_1, A_2, \dots, A_n$ e dei domini di tali attributi $D_1, D_2, \dots, D_n$ forma il cosiddetto schema di relazione, che si denota come $R(A_1 : D_1, A_2 : D_2, \dots, A_n : D_n)$ di conseguenza una tupla di R si può denotare anche come $\langle A_1 : a, A_2 : b, \dots, A_n : c \rangle$

Una tupla di una relazione si può rappresentare come **tupla etichettata**, in cui le etichette sono gli attributi della relazione, ed i valori associati alle etichette sono i valori in corrispondenza dei vari attributi.

Esempio su tabella di prima:

tupla etichettata $t1 = \langle Casa : \text{'Juve'}, Fuori : \text{'Lazio'}, RetiCasa : 3, RetiFuori : 1 \rangle$

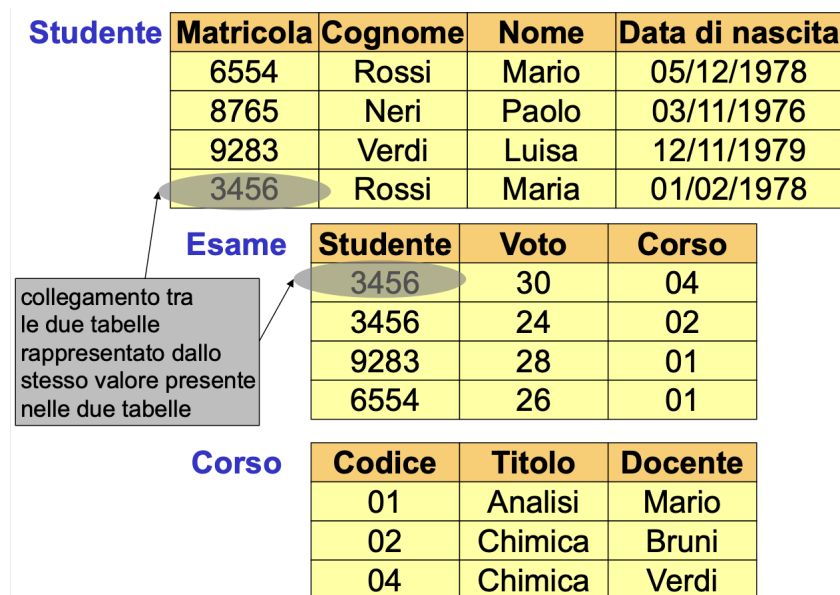


Figura 1.3: caption

Schema e istanze

Schema di relazione: un nome di relazione R con un insieme non vuoto di attributi $A_1, \dots, A_n$ ed eventualmente anche i corrispondenti domini $R(A_1, \dots, A_n)$

Istanza di relazione su uno schema $R(X)$: insieme r delle tuple sull'insieme di attributi X

Relazione: schema T di relazione + istanza su T

Schema di base di dati: insieme non vuoto di schemi di relazione $R = R_1(A_1A_2\dots A_n), \dots, R_m(B_1B_2\dots B_p)$

Istanza di base di dati su uno schema $R = \{R_1(X_1), \dots, R_m(X_m)\}$: insieme di relazioni $r = \{r_1, \dots, r_m\}$, dove per ogni $i = 1, \dots, m, r_i$ è una istanza relazione sullo schema $R_i(X_i)$

Base di dati: schema S di basi di dati + istanza di basi di dati su S

Algebra relazionale

I suoi operatori sono definiti su relazioni, producono relazioni e possono essere composti.

Operatori dell'algebra relazionale:

- **insiemistici:** unione, intersezione, differenza
- **ridenominazione**
- **selezione**
- **proiezione**
- **join in diverse versioni:** join naturale, prodotto cartesiano, theta-join

Operatori insiemistici

Le relazioni sono insiemi di tuple quindi è sensato definire per esse gli operandi insiemistici, sono operatori binari e i risultati dell'applicazione di tali operandi sono a loro volta relazioni.

Si può applicare **unione**, **intersezione**, **differenza** solo a relazioni che hanno uguali definizioni intensionali

Unione

Laureato

Matricola	Nome	Età
7274	Rossi	42
7432	Neri	54
9824	Verdi	45

Dirigente

Matricola	Nome	Età
9297	Neri	33
7432	Neri	54
9824	Verdi	45

Laureato $\cup$ Dirigente

Matricola	Nome	Età
7274	Rossi	42
7432	Neri	54
9824	Verdi	45
9297	Neri	33

schema dell'unione

tuple da Laureato

tuple da Dirigente

Intersezione

Laureato

Matricola	Nome	Età
7274	Rossi	42
7432	Neri	54
9824	Verdi	45

Dirigente

Matricola	Nome	Età
9297	Neri	33
7432	Neri	54
9824	Verdi	45

Laureato $\cap$ Dirigente

Matricola	Nome	Età
7432	Neri	54
9824	Verdi	45

schema dell'intersezione

tuple da Laureato e Dirigente

Differenza

Laureato

Matricola	Nome	Età
7274	Rossi	42
7432	Neri	54
9824	Verdi	45

Dirigente

Matricola	Nome	Età
9297	Neri	33
7432	Neri	54
9824	Verdi	45

Laureato – Dirigente

Matricola	Nome	Età
7274	Rossi	42

schema della differenza

tuple di Laureato

Ridenominazione

Un'unione sensata ma impossibile in algebra: vogliamo i genitori ed i loro figli

Paternità

Padre	Figlio
Adamo	Abele
Adamo	Caino
Abramo	Isacco

Maternità

Madre	Figlio
Eva	Abele
Eva	Set
Sara	Isacco

Paternità $\cup$ **Maternità**
??

impossibile
eseguire l'unione:
le due relazioni
hanno differenti
insiemi di attributi

modifica lo schema cambiando il nome di uno o più attributi lasciando inalterata l'istanza dell'operando

Paternità

Padre	Figlio
Adamo	Abele
Adamo	Caino
Abramo	Isacco

$REN_{Genitore \leftarrow Padre}$ (Paternità)

Genitore	Figlio
Adamo	Abele
Adamo	Caino
Abramo	Isacco

ridenomina l'attributo
"Padre" in "Genitore"

Ridenominazione: sintassi e semantica

Sintassi:

$$REN_{A_1, A_2, \dots, A_n \leftarrow B_1, B_2, \dots, B_n} (Operando)$$

Semantica:

Lo schema della relazione rappresentata da "Operando" viene modificato sostituendo al nome di attributo B_1 il nome A_1 , al nome di attributo B_2 il nome A_2 , ..., e al nome di attributo B_n il nome di attributo A_n

Paternità

Padre	Figlio
Adamo	Abele
Adamo	Caino
Abramo	Isacco

$REN_{Genitore \leftarrow Padre}$ (Paternità)

Genitore	Figlio
Adamo	Abele
Adamo	Caino
Abramo	Isacco

$REN_{Genitore \leftarrow Padre}$ (Paternità)

$\cup$

$REN_{Genitore \leftarrow Madre}$ (Maternità)

Maternità

Madre	Figlio
Eva	Abele
Eva	Set
Sara	Isacco

$REN_{Genitore \leftarrow Madre}$ (Maternità)

Genitore	Figlio
Eva	Abele
Eva	Set
Sara	Isacco

Genitore	Figlio
Adamo	Abele
Adamo	Caino
Abramo	Isacco
Eva	Abele
Eva	Set
Sara	Isacco

selezione

Operatore monadico (un solo argomento) che prevede la formulazione di condizione, produce un risultato che:

- ha lo stesso schema dell'operando
- contiene un sottoinsieme delle tuple dell'operando, quelle che soddisfano la condizione. espressamente indicata nell'operatore

Impiegato

Matricola	Cognome	Filiale	Stipendio
7309	Rossi	Roma	55
5998	Neri	Milano	64
9553	Milano	Milano	44
5698	Neri	Napoli	64

impiegati che

- guadagnano più di 50
- guadagnano più di 50 e lavorano a Milano
- hanno lo stesso nome della filiale presso cui lavorano

Selezione: sintassi e semantica

Sintassi:

$$SEL_{Condizione}(Operando)$$

Condizione: espressione booleana in cui gli operandi utilizzano gli attributi delle relazioni in gioco

Semantica:

la relazione risultato ha gli stessi attributi dell'operando e contiene le tuple dell'operando che soddisfano la condizione specificata

esempio: impiegati che guadagnano più di 50

Impiegato

Matricola	Cognome	Filiale	Stipendio
7309	Rossi	Roma	55
5998	Neri	Milano	64
5698	Neri	Napoli	64

risultato

non soddisfa
la condizione

$$SEL_{Stipendio > 50}(Impiegato)$$

impiegati che guadagnano più di 50 e lavorano a Milano

Impiegato

Matricola	Cognome	Filiale	Stipendio
5998	Neri	Milano	64

$$SEL_{Stipendio > 50 \text{ AND } Filiale = 'Milano'}(Impiegato)$$

Proiezione

Operatore monadico, che produce un risultato che:

- ha parte degli attributi dell'operando
- contiene tuple cui contribuiscono tutte le tuple dell'operando: sono le tuple ottenute dall'operando eliminando gli attributi che non compaiono nella lista espressamente indicata nell'operatore

Impiegato

Matricola	Cognome	Filiale	Stipendio
7309	Rossi	Roma	55
5998	Neri	Milano	64
9553	Milano	Milano	44
5698	Neri	Napoli	64

per tutti gli impiegati:

- matricola e cognome
- cognome e filiale

Proiezione: sintassi e semantica

Sintassi:

$$PROJ_{ListaAttributi}(Operando)$$

Semantica:

la relazione risultato ha i soli attributi contenuti in ListaAttributi, e contiene le tuple ottenute da tutte le tuple dell'operando ristrette agli attributi nella lista

matricola e cognome di tutti gli impiegati

Impiegato

Matricola	Cognome
7309	Neri
5998	Neri
9553	Rossi
5698	Rossi

$$PROJ_{Matricola,Cognome}(Impiegato)$$

Selezione e Proiezione

Combinando selezione e proiezione, possiamo estrarre interessanti informazioni da una relazione

Restituire matricola e cognome degli impiegati che guadagnano più di 50

Impiegato

Matricola	Cognome
7309	Rossi
5998	Neri
5698	Neri

$PROJ_{Matricola, Cognome}(SEL_{Stipendio > 50}(Impiegato))$

La negazione in algebra relazionale

Per esprimere nel linguaggio dell'algebra condizioni di negazione è utilizzare l'operatore di differenza. Il corrispettivo logico di questo principio è la seguente regola per scrivere query corrette:

l'insieme degli elementi di A che non soddisfano la proprietà B è uguale all'insieme che otteniamo da A togliendo gli elementi che soddisfano la proprietà B:

$$\{x | x \in A \wedge \neg B(x) = A - \{x | B(x)\}$$

esempio:

Concerto(idconc, idart, anno, città)

Artista(idart, nome, tipo)

Evento(ideve, idconc, voto)

richiesta: calcolare l'idconc dei concerti che non sono stati svolti nell'ambito di un evento

$PROJ_{idconc}(Concerto) - PROJ_{idconc}(Evento)$

Join

Combinando selezione e proiezione, possiamo estrarre informazioni da una relazione, non possiamo però correlare informazioni presenti in relazioni diverse. Esistono diversi tipi di join: join naturale, theta-join, quijoin, join esterno ecc.

join naturale

Operatore binario, produce un risultato:

- il cui schema ha l'unione degli attributi degli operandi
- le cui tuple sono costruite ciascuna a partire da due tuple che si combinano, una di un operando ed una di un altro operando

Join naturale: sintassi e semantica

Siano $R_1(X_1), R_2(X_2)$ con X_1 e X_2 che sono due insiemi, $R_1 \text{ JOIN } R_2$ è una relazione su $X_1 X_2$ il cui insieme di tuple è:

$\{t \text{ su } X_1 X_2 | \text{ esistono due tuple } t_1 \in R_1 \text{ e } t_2 \in R_2 \text{ tali che } t[X_1] = t_1 \text{ e } t[X_2] = t_2\}$

$R_1 \text{ JOIN } R_2$ è una relazione il cui insieme di tuple contiene ogni tupla t tale che esistono due tuple $T_1 \in R_1$ e $t_2 \in R_2$ tali che

- t negli attributi X_1 ha gli stessi valori di t_1
- t negli attributi X_2 ha gli stessi valori di t_2

Una tupla t esiste solo se le due tuple t_1 e t_2 hanno gli stessi valori negli attributi che R_1 e R_2 hanno in comune. Diciamo che $t_1 \in R_1$ e $t_2 \in R_2$ sono combinabili dal join naturale se $t_1[X_1 \cap X_2] = t_2[X_1 \cap X_2]$. Dalla definizione segue che ogni tupla nel join tra R_1 ed R_2 proviene da due tuple combinabili dal join. In effetti $t_1 \in R_1$ e $t_2 \in R_2$ tali che $t_1[X_1 \cap X_2] = t_2[X_1 \cap X_2]$ si combinano per ottenere la tupla t tale che $t[X_1] = t_1$ e $t[X_2] = t_2$

Insegna		Join naturale: esempio		Tratta	
Docente	Corso	Corso	Ling		
1	BD	BD	SQL	t1	t2
2	PS	BD	Java		
3	Reti	PS	Java		
1	PS	PS	UML		

Insegna join Tratta		Docente	Corso	Ling
t	t1	1	BD	SQL
		1	BD	Java
		1	PS	Java
		1	PS	UML
		2	PS	Java
		2	PS	UML

L'insieme degli attributi del risultato è $\{Docente, Corso, Ling\}$
 $=$
 $\{Docente, Corso\} \cup \{Corso, Ling\}$

nella tabella con il join non è presente il corso Reti perchè non è in comune

Join completo

Un join completo è un join in cui ogni tupla contribuisce al risultato.

Impiegato	Reparto	Reparto	Capo
Rossi	A	A	Mori
Neri	B	B	Bruni
Bianchi	B		

Impiegato	Reparto	Capo
Rossi	A	Mori
Neri	B	Bruni
Bianchi	B	Bruni

Join il cui risultato è l'insieme vuoto di tuple

Impiegato	Reparto	Reparto	Capo
Rossi	A	D	Mori
Neri	B	C	Bruni
Bianchi	B		

Impiegato	Reparto	Capo

Theta-join

Siano R_1 ed R_2 due relazioni senza attributi in comune tali quindi che il loro join naturale corrisponda al prodotto cartesiano. Ma nella pratica il prodotto cartesiano ha senso solo se combinato con una relazione:

$$SEL_{condizione}(R_1 \text{ JOIN } R_2)$$

La combinazione delle due operazioni viene chiamata **theta-join**, richiede appunto che R_1 ed R_2 non abbiano attributi in comune e viene indicata con

$$R_1 \text{ JOIN}_{\text{condizione}} R_2$$

Self-join

Il **self-join** è un equi-join in cui la stessa relazione compare sia come operando sinistro sia come operando destro, ovvero, quando una operazione di join coinvolge la stessa relazione due volte. Siccome l'equi-join prevede che i due operandi non abbiano attributi in comune, quando si applica un self-join in uno dei due operandi occorre ridenominare tutti gli attributi della relazione.

Riassunto join

- **Join naturale** tra R_1 e R_2 (si può eseguire sempre)
 - due tuple si accoppiano se e solo se hanno gli stessi valori negli attributi che R_1 ed R_2 hanno in comune
 - se R_1 ed R_2 non hanno attributi in comune, il risultato del join naturale è il prodotto cartesiano tra due relazioni perchè ogni tupla di R_1
 - si combina con ogni tupla di R_2 per dare origine ad una tupla nel risultato
- **Theta-join** tra R_1 e R_2 (richiede che R_1 e R_2 non abbiano attributi in comune)
 - due tuple si accoppiano se la condizione di join è soddisfatta dalle due tuple
- **Equi-join** tra R_1 e R_2 (richiede che R_1 e R_2 non abbiano attributi in comune)
 - è un theta-join in cui la condizione di join è una uguaglianza tra un attributo di R_1 ed un attributo di R_2

Esercizi query

Tutte le seguenti query sono effettuate sul seguente database:

Concerto(idconc, idart, anno, città)

Artista(idart, nome, tipo)

Evento(ideve, idconc, voto)

query1: trovare nome e tipo degli artisti che hanno tenuto almeno un concerto prima del 2000

soluzione: nome e tipo degli artisti li troviamo nella relazione Artista, mentre i concerti dell'artista li troviamo nella relazione Concerto. Ne segue che per rispondere alla richiesta, la query deve esplorare il legame tra Artista e Concerto e questo si realizza attraverso il join naturale tra le sue due relazioni.

$$PROJ_{\text{nome, tipo}}(\text{Artista JOIN } SEL_{\text{anno} < 2000}(\text{Concerto}))$$

query2: trovare l'id e nome degli artisti che hanno tenuto almeno un concerto svoltosi in una città diversa da Roma nell'ambito di un evento nel quale ha ottenuto un voto maggiore di 4

soluzione: eseguo due join naturali, il primo tra Artista e Concerto su attributo comune idart, il secondo tra Concerto e Evento su attributo comune idconc.

$$PROJ_{\text{idart, nome}}((\text{Artista JOIN } SEL_{\text{città} < > 'Roma'}(\text{Concerto})) \text{ JOIN } SEL_{\text{voto} > 4}(\text{Evento}))$$

query3: trovare l'id degli artisti che hanno tenuto almeno due concerti nella stessa città in anni diversi.

soluzione: poichè dobbiamo tentare di collegare due tuple della stessa relazione (due concerti tenuti nella stessa città), si deve calcolare il join di Concerto con se stesso, ovviamente stando attenti alle ridenominazioni

$$PROJ_{\text{idart}}(REN_{i2 \leftarrow \text{idconc}, t1 \leftarrow \text{idart}, a1 \leftarrow \text{anno}, c1 \leftarrow \text{città}}(\text{Concerto}) \text{ JOIN}_{\text{idart} = t1 \text{ and } \text{città} = c1 \text{ and } \text{anno} < > a1} \text{Concerto})$$

query3: trovare l'id degli artisti che dopo il 2010 hanno tenuto concerti **solo** a Roma soluzione: dobbiamo capire come intendere "solo", "hanno tenuto concerti solo a Roma" ha lo stesso significato di "**non** hanno tenuto concerti in una città diversa da Roma".

Quindi per calcolare questa query dobbiamo togliere da tutti gli artisti quelli che dopo il 2010 hanno tenuto

almeno un concerto in una città diversa da Roma:

$$PROF_{idart}(Artista) - PROJ_{idart}(SEL_{citta<>Roma \text{ and } anno>2000}(Concerto))$$

Capitolo 2

SQL